

What is package?

- ◆ A package is just like a container that contains collection classes.
- ◆ Java uses package to group related **classes**, **interfaces** and **sub-packages** in any java project.
- ◆ We can assume package as a folder or a directory that is used to store similar files.
- ◆ Ex.

```
package mypack;  
public class employee  
{  
    String empId;  
    String name;  
}
```

- ◆ In previous statement will create a package with name **mypack** in the project directory.

TYPES

predefined

- java.lang
- java.util
- java.io
- java.applet
- java.awt
- java.net
- java.SQL

User-defined

- package pi
- package add
- package kiara

- ◆ **java.lang** is the default package in java. The classes and interfaces related to data structure which is included in **java.util** package.
- ◆ The classes/package under in file handling which is included in **java.io** package.
- ◆ **java.applet**, **java.awt**, **java.net** packages are used for web applications.
- ◆ To import java package into a class, we need to use java **import** keyword. which is used to access package and its classes into the java program.

PACKAGES IN JAVA

A package in java is a group of related classes, interfaces and sub-packages. It is used to organize code, avoid naming conflicts and control access.

1 WHY PACKAGES?

- Organizes related classes and interfaces.
- Avoids naming conflicts between classes.
- Provides access protection.
- Makes maintenance and reuse of code easier.
- Built-in packages provide ready to use classes.

3 TYPES OF PACKAGES

- Built-in (Standard) Packages
 - ↳ Provided by Java API.
 - Example: java.lang, java.util, java.io, java.sql
- User Defined Packages
 - ↳ Created by developers.
 - ↳ Used to organize user-defined classes.

4 HOW TO CREATE A PACKAGE

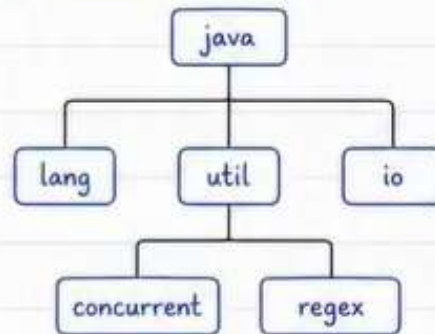
```
// Save this file as: com/example/demo/Hello.java
package com.example.demo; // package declaration

public class Hello {
    public void show() {
        System.out.println("Hello from package!");
    }
}
```

5 ACCESSING CLASSES FROM A PACKAGE

- From the same package
 - ↳ No import is required.
- ```
package com.example.demo;
public class Test {
 Hello h = new Hello(); // same package
}
```
- From a different package
    - ↳ Use import statement.
- ```
package com.example.app;
import com.example.demo.Hello;
public class Test {
    Hello h = new Hello(); // imported class
}
```

2 PACKAGE HIERARCHY



java is the root package.
All other packages are sub-packages.

@code.withh.clarity

STEPS:

- 1 Create directory structure as per package name (com/example/demo).
- 2 Write package statement as the first line in the source file.
- 3 Save the file inside that directory.

6 IMPORT STATEMENT

- Used to access classes from another package.
- Types:
 - ↳ Import specific class

```
import java.util.ArrayList;
```
 - ↳ Import all classes from a package

```
import java.util.*;
```

7 KEY POINTS

- ✓ package statement must be the first statement in the source file.
- ✓ The package name should be all lowercase.
- ✓ Classes in a package can access each other without import.
- ✓ Use meaningful package names (e.g., com.company.project.module).



SUMMARY

Packages help in organizing code, avoiding conflicts, improving access control and enhancing code reusability.



Built-in Packages in Java



Java provides a rich set of built-in packages that contain classes and interfaces.

These packages are part of Java API.

Important Built-in Packages

- ① **java.lang** - Contains fundamental classes.
(String, Math, System, Object, etc.)
- ② **java.util** - Contains utility classes and interfaces.
(ArrayList, HashMap, Date, Scanner, etc.)
- ③ **java.io** - For input and output operations.
(File, InputStream, OutputStream, etc.)
- ④ **java.net** - For networking and URL handling.
(Socket, ServerSocket, URL, etc.)
- ⑤ **java.sql** - For database connectivity.
(Connection, Statement, ResultSet, etc.)
- ⑥ **java.time** - For date and time API.
(LocalDate, LocalTime, ZonedDateTime, etc.)
- ⑦ **java.math** - For mathematical operations.
(BigInteger, BigDecimal, etc.)
- ⑧ **java.text** - For formatting and parsing text.
(SimpleDateFormat, NumberFormat, etc.)
- ⑨ **java.nio** - For New Input/Output operations.
(Buffer, Channel, Charset, etc.)
- ⑩ **java.awt** - For GUI components and graphics
(Button, Frame, Color, Graphics, etc.)
- ⑪ **javax.swing** - For advanced GUI components.
(JFrame, JButton, JLabel, etc.)

Note:

To use any built-in package, we need to import it using import keyword.



Example:

```
import java.util.*;  
import java.io.*;  
  
// use classes  
from the  
imported package
```



Key Takeaway:

Built-in packages make Java powerful by providing ready-to-use classes and interfaces.

#Java #Packages
#Javaapi #Coding

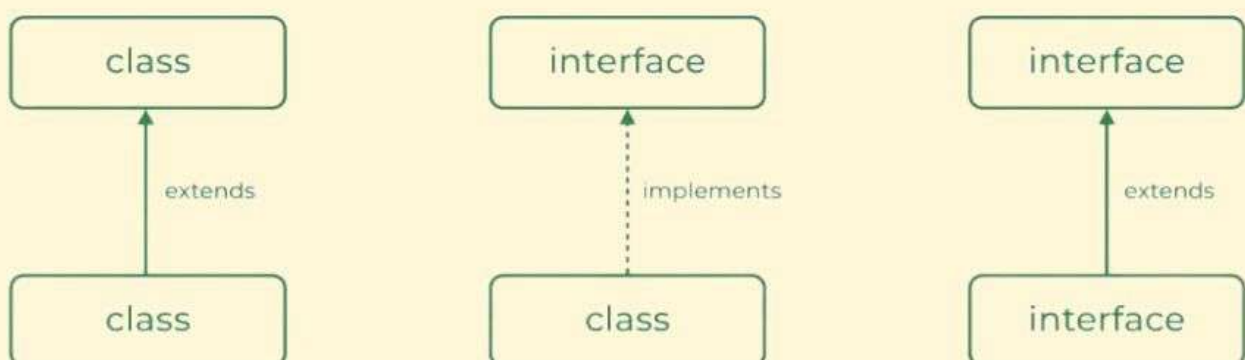


INTERFACE

What is Interface :

- An Interface in Java is an abstract type that defines a set of methods a class must implement.
- It is used to achieve abstraction and multiple inheritance in Java. We define interfaces for capabilities (e.g., Comparable, Serializable, Drawable).
- A class that implements an interface must implement all the methods of the interface. Only variables are public static final by default.

Relationship Between Class and Interface:





JAVA INTERFACES AND CLASSES

Similarities with classes :

- An interface can contain any number of methods.
- An interface is written in a file with a .java extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in a .class file.
- Interfaces appear in packages and their corresponding bytecode file must be in a directory structure that matches the package name.

Differences with classes :

- You cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class, It is implemented by a class.
- An interface can extend multiple interfaces.





IMPLEMENTING INTERFACES

- To implement an interface, we use the keyword **implements**

```
/* File name : MammalInt.java */
public class MammalInt implements Animal {
    public void eat() {
        System.out.println("Mammal eats");
    }

    public void travel() {
        System.out.println("Mammal travels");
    }

    public int noOfLegs() {
        return 0;
    }

    public static void main(String args[]) {
        MammalInt m = new MammalInt();
        m.eat();
        m.travel();
    }
}

interface Animal {
    public void eat();
    public void travel();
}
```





EXTENDING JAVA INTERFACES

- An interface can extend another interface in the same way that a class can extend another class. The `extends` keyword is used to extend an interface, and the child interface inherits the methods of the parent interface.

```
// Filename: Sports.java
public interface Sports {
    public void setHomeTeam(String name);
    public void setVisitingTeam(String name);
}

// Filename: Football.java
public interface Football extends Sports {
    public void homeTeamScored(int points);
    public void visitingTeamScored(int points);
    public void endOfQuarter(int quarter);
}

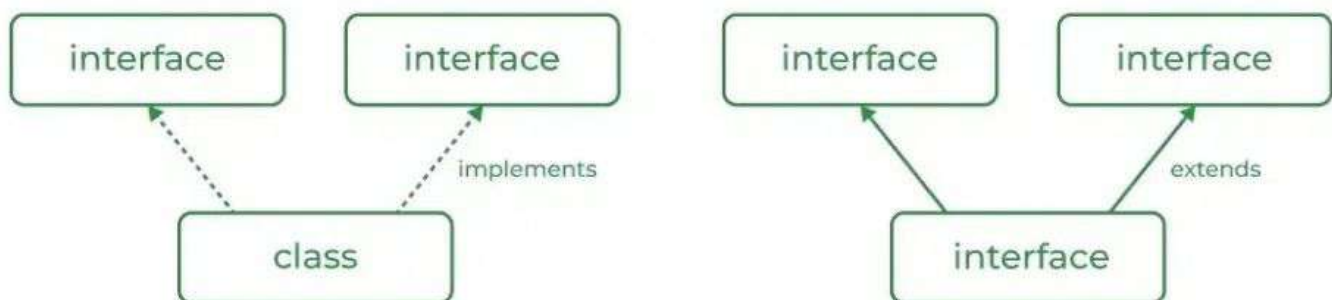
// Filename: Hockey.java
public interface Hockey extends Sports {
    public void homeGoalScored();
    public void visitingGoalScored();
    public void endOfPeriod(int period);
    public void overtimePeriod(int ot);
}
```



MULTIPLE INHERITANCE IN JAVA

- Java does not support multiple inheritance with classes to avoid ambiguity, but it supports multiple inheritance using interfaces.

Multiple inheritance in Java



What is an interface ?

- Interface is like a contract that defines a set of methods that a class must implement
- It is a way to specify what methods a class should have, without providing the actual implementation for those methods

Key Points about Interfaces

1. Abstract Methods
2. Default Methods
3. Static Methods
4. Implementation
5. Multiple Inheritance
6. Constants

1. Abstract Methods

- All methods in an interface are abstract (unless they are default or static methods)
- This means that they do not have a body and must be implemented by any class that implements the interface



```
interface Animal {  
    void eat();  
    void sleep();  
}
```

2. Default Methods

- Default methods in an interface have a body and provide a default implementation
- Classes can override them if needed

```
interface Animal {  
    void eat();  
    void sleep();  
  
    default void makeSound() {  
        System.out.println("Some sound");  
    }  
}
```

3. Static Methods

- Static methods in an interface have a body and can be called without an instance of the interface
- They cannot be overridden by implementing classes

```
interface Animal {  
    void eat();  
    void sleep();  
  
    static void info() {  
        System.out.println("Animals are living beings.");  
    }  
}
```

4. Implementation

- A class that implements an interface must provide implementations for all of its abstract methods

```
class Dog implements Animal {  
    @Override  
    public void eat() {  
        System.out.println("Dog eats.");  
    }  
  
    @Override  
    public void sleep() {  
        System.out.println("Dog sleeps.");  
    }  
}
```

5. Multiple Inheritance

A class can implement multiple interfaces, allowing Java to provide a form of multiple inheritance

```
interface Animal {  
    void eat();  
    void sleep();  
}  
  
interface Pet {  
    void play();  
}  
  
class Dog implements Animal, Pet {  
    @Override  
    public void eat() {  
        System.out.println("Dog eats.");  
    }  
  
    @Override  
    public void sleep() {  
        System.out.println("Dog sleeps.");  
    }  
  
    @Override  
    public void play() {  
        System.out.println("Dog plays.");  
    }  
}
```

5. Constants

Any variables declared in an interface are implicitly public, static, and final

```
interface Animal {  
    // implicitly public, static, final  
    int NUMBER_OF_LEGS = 4;  
  
    void eat();  
    void sleep();  
}
```

Practical Examples

Creating API Specifications

Interfaces are often used to define the contracts for APIs

```
interface PaymentProcessor {
    void processPayment(double amount);
}

class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("credit card payment of $" + amount);
    }
}
```

```
class PayPalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("PayPal payment of $" + amount);
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        PaymentProcessor ccProcessor = new CreditCardProcessor();
        PaymentProcessor ppProcessor = new PayPalProcessor();

        ccProcessor.processPayment(100.00);
        ppProcessor.processPayment(200.00);
    }
}
```

Practical Examples Creating API Specifications

Using Interfaces for Dependency Injection

```
interface MessageService {  
    void sendMessage(String message);  
}  
  
class EmailService implements MessageService {  
    @Override  
    public void sendMessage(String message) {  
        System.out.println("Email message: " + message);  
    }  
}
```

```
class MyApplication {  
    private MessageService messageService;  
  
    // Constructor injection  
    public MyApplication(MessageService service) {  
        this.messageService = service;  
    }  
  
    public void processMessages(String message) {  
        messageService.sendMessage(message);  
    }  
}
```